Batch: iitr-as-260313 · **Program:** Certification Program in Agentic Systems and Design Offline Exam

This guide compiles the key concepts you need for your upcoming evaluation. Work through each section, focus on the worked examples and definitions, and use the key terms to self-test. (Content for one lecture ID in this set was unavailable in the source system and is not covered below.)

1. Python Fundamentals

1.1 Environment Setup & Tooling

- **VS Code**: install per OS, open the correct project folder, enable auto-save.
- **Python install**: add Python to system PATH; install on local disk (not OneDrive) to avoid path errors; use an online compiler as a fallback.
- Run scripts via VS Code terminal (`python3 filename.py`) or an online compiler.
- **GitHub**: used to store/share code as a portfolio (see §2).
- AI code-completion extensions (e.g. Augment) assist coding; treat as aids, not crutches — self-research and independent debugging are explicitly emphasized as job-critical skills.

Key terms: PATH, terminal, virtual environment, portfolio repo.

1.2 Variables, Data Types & Operators

- **Variables**: named containers (`age = 26`); reassignable; printing shows current value.
- **Four basic types**: `int`, `float`, `str`, `bool`. Check with `type(x)`. Python is **case-sensitive** (`True` ≠ `true`).
- **Arithmetic operators**: `+ - * / // % **`. `/` is true division (returns float); `//` is floor division (returns int).
- **Operator precedence (BODMAS)**: Brackets > Exponents > Mult/Div > Add/Sub; use parentheses to force order.
- **Comparison operators**: `== != > < >= <=` → always return `bool`. `=` is assignment, `==` is equality — a very common bug source.
- **String concatenation**: `+` joins strings; joining a string and a number directly raises an error — explicit conversion (`str()`/`int()`/`float()`) is required.
- Float precision compares only up to ~15 decimal digits.
- Comments start with `#`.

1.3 Logical Operators & Conditionals

- **Logical operators**: `and` (all true), `or` (any true), `not` (negate). Precedence: `()` > `not` > `and` > `or`.
- **Short-circuit evaluation**: `and` skips the 2nd operand if the 1st is `False`; `or` skips the 2nd if the 1st is `True`.
- **`if` / `elif` / `else`**: only one branch of a chain executes; indentation (4 spaces) defines blocks; nested `if`s allow layered checks.

- **Expressions vs statements**: an expression evaluates to a value (`2 + 3`); a statement performs an action (`x = 5`).
- **Compound assignment**: `a += 10` ≡ `a = a + 10` (also `-=`, `*=`, `/=`).

1.4 Input/Output & String Formatting

- `input()` always returns a **string**; convert with `int()`/`float()` as needed. `bool("")` → `False`; `bool()` of any non-empty string → `True`.
- `print()` supports multiple args, `sep=` (default space) and `end=` (default newline).
- **f-strings**: `f"Hello {name}, {age} yrs"`; format specifiers e.g. `{pi:.2f}` (2 decimals), `{num:,}` (thousands separator).

1.5 Loops

- **`while condition:`** — runs while true; must update the loop variable to avoid an infinite loop; used for input validation.
- **`for var in sequence:`** — iterates over a sequence or `range()`.
  - `range(stop)`, `range(start, stop)`, `range(start, stop, step)` — `stop` is exclusive.
- **`break`** exits the loop entirely; **`continue`** skips only the current iteration.
- **Nested loops**: total iterations = outer × inner; outer loop = rows, inner = columns; used for pattern printing (`end=''` to stay on one line).

1.6 Lists

- **Ordered, heterogeneous, dynamic** containers; `[]` or `list()`. Zero-based indexing; negative indices count from the end.
- **Slicing**: `list[start:stop:step]` — `start` inclusive, `stop` exclusive.
- **Iteration**: `for item in list`, `for i in range(len(list))`, `for i, item in enumerate(list)`.
- **Common methods**: `len()`, `sum()`, `sorted()` (returns new list) vs `.sort()` (in place), `.reverse()`, `.append()` (1 item) vs `.extend()` (multiple), `.pop()`/`.remove()`, `.count()`, `.index()`.
- `list()` converts any iterable; `zip()` pairs multiple iterables element-wise.
- ML relevance: lists commonly hold datasets, batches, and hyperparameter sweeps because they're dynamic and heterogeneous.

1.7 Strings & Dictionaries

- **Strings**: immutable; single/double/triple (multi-line) quotes; indexing, `+` concat, `*` repetition, `len()`, `.upper()`/`.lower()`/`.title()`/`.capitalize()`.
- **Dictionaries**: key-value pairs; keys unique (duplicate keys overwrite); `{}`/`dict()`.
  - Access: `d[key]` raises `KeyError` if missing; `d.get(key, default)` is the safe form.
  - `.keys()`, `.values()`, `.items()` for iteration; `in` checks key existence; supports nested dicts.

1.8 Functions

- `def name(params): ... return value`. `return` sends a value back and ends execution (enables chaining); `print` only displays — it does not affect control flow or return value.
- **Parameter types**: positional (required, in order), default (fallback value), keyword (named at call site), variable-length (`*args`/`**kwargs`).
- **Scope**: global (outer, program-wide) vs local (inside function only); local shadows global with the same name.
- **Pitfall**: never use a mutable object (e.g. a list) as a default parameter value — it's shared across calls; use `None` and initialize inside the function instead.
- Functions should do one clearly-defined task (reusability, maintainability); in ML they encapsulate model/pipeline steps.

1.9 Classes & Objects (OOP)

- **Class** = blueprint (PascalCase name) combining data (state) + behavior (methods).
- **Object** = an instance; independent state per object even from the same class.
- `__init__` = constructor, auto-runs on instantiation; `self` refers to the current instance and accesses its attributes/methods.
- Example pattern: a `BankAccount` class with `owner`, `balance`, and `deposit()`/`withdraw()` methods that check conditions before updating state.

Key terms: immutability, KeyError, `self`, constructor, mutable default argument.

---

2. Version Control: Git & GitHub

- **Why version control**: prevents conflicting simultaneous edits; tracks who changed what, when, why.
- **VCS types**: Local (single machine, risky) → Centralized (single server, single point of failure) → **Distributed** (Git: full history on every machine + remote — most redundant).
- **Git** = local VCS software; **GitHub** = cloud platform to host/share Git repos ("Instagram for projects").
- **Three-area model**: Working Directory → Staging Area → Local Repository.

Command	Purpose
`git status`	Show tracked/untracked/changed files
`git add <file>` / `git add *`	Stage files
`git commit -m "msg"`	Commit staged changes
`git push` / `git pull`	Sync local ↔ remote
`git clone <url>`	Copy a remote repo locally
`git log`	Commit history

Command	Purpose
`git diff`	Show changes
`git restore <file>`	Discard uncommitted changes
`git reset <file>`	Unstage changes
`git commit --amend -m "msg"`	Edit latest commit message
`git switch`/`git checkout <branch>`	Change branch
`git branch -d <branch>`	Delete a local branch

- **Branching**: `main`/`master` holds stable code; feature branches isolate new work; a **Pull Request (PR)** requests review before merging; remote branches show as `origin/<name>`.
- **End-to-end flow**: create repo → clone → branch → commit → push → PR → merge → delete feature branch.
- **Access roles**: Read (view) < Write (push) < Maintain (issues/PRs/settings) < Admin (full control).

Key terms: staging area, pull request, distributed VCS, `origin/`.

---

3. Foundations of AI Agents

3.1 What Is an Agent

- An **AI agent** perceives its environment, reasons/decides, and acts autonomously toward a goal — unlike traditional rule-based systems that follow fixed workflows with no decision-making.
- **Core components**: **Planning** (breaks a goal into ordered subtasks, handles fallback/retry), **Reasoning** (evaluates options given context), **Memory** (short-term = session-only; long-term = across sessions), **Action** (executes real-world tasks — ordering, payments, messages, API calls; sensitive actions may need human approval).
- **Working loop**: **Observe → Think (Plan + Reason) → Act**, repeating until the goal is met.
- **Characteristics**: Autonomy, Goal-Orientation, Adaptability, Task Decomposition.
- **Reliability**: agents are typically ~90–95% reliable; failures stem from missing data, memory/DB outages, or bad reasoning — hence human-in-the-loop for critical actions.
- Agent = **LLM (brain) + Tools (workers)**: the LLM reasons/orchestrates; tools execute (search, DB queries, code execution, third-party APIs).

Key terms: goal decomposition, fallback mechanism, human-in-the-loop, context window.

---

4. Large Language Models (LLMs)

4.1 What LLMs Are

- Trained on massive text corpora to predict the next word/token and generate human-like text. Examples: GPT, Gemini, Claude, Llama.
- Strengths: context/tone understanding, role adoption, multi-format output (text/code/tables/JSON/PDF), summarization, translation, Q&A, classification.
- **Limitations**: **hallucination** (confident but wrong output, worse in long/complex contexts); static knowledge (frozen at training cutoff — needs live tools/RAG for current data); weak native math reasoning; limited **context window**; probabilistic (non-deterministic) outputs.

4.2 Tokenization & Transformers

- **Tokens** are text units (~0.75 words/token on average); both input and output tokens count toward cost/latency.
- **Transformer architecture** (Google, 2017) processes an entire sequence at once (vs older RNNs that process sequentially), preserving long-range context — the backbone of modern LLMs.

Key terms: hallucination, context window, token, transformer.

---

5. Prompt Engineering

5.1 Prompt Basics

- A **prompt** is any input to the model. **Prompt engineering** = crafting prompts for accurate, useful output.
- **System prompt** (set by developers: persona, tone, boundaries, format, scope — given priority by design, but a user can still bypass it via prompt injection, so guardrails are still needed) vs **user prompt** (the end-user's input, optimizable).
- **Role prompting**: assigning a persona ("You are a biology professor…") sharpens the response register/detail level.

5.2 Structured Prompt Frameworks

- **CIFC**: **C**ontext (who/what the AI is) · **I**nstruction (the task) · **F**ormat (output shape: bullets/JSON/table) · **C**onstraints (boundaries).
- **Structured prompt components** (equivalent, extended): Role, Task, Instructions, Constraints, Output Format.
- **CICO/SECO validation**: Clarity, Intent, Constraints, Output — a checklist to audit any prompt before use.
- Placing critical instructions at the **start and end** of a prompt gets more model attention; be explicit and positive rather than vague/negative; few-shot examples improve output quality.

5.3 Chain of Thought (CoT) & Reasoning Prompts

- **CoT** asks the model to reason step-by-step rather than jump to a final answer, reducing errors on multi-step tasks.
  - **Zero-shot CoT**: add "think step by step."
  - **Few-shot CoT**: provide worked reasoning examples first.
- **Reasoning prompts**: used for comparisons/decisions (e.g. "Python vs JavaScript for X") — give criteria, ask for justified recommendations.

5.4 Self-Reflection, Self-Correction & Feedback Loops

- **Self-reflection**: the model generates an intermediate answer, checks it against stated criteria, and only then emits a refined final answer.
- **Self-correction (3-stage)**: (1) generate → (2) critique (find gaps) → (3) rewrite/fix. Intermediate stages stay hidden from the user.
- **Iterative prompting**: breaks a large task into phases, each refined by follow-up prompts — needed when one shot can't capture all requirements.
- **Feedback loops**: **User feedback loop** (explicit human corrections) vs **Internal evaluation loop** (the AI scores its own output against an evaluation file/criteria and revises until the threshold is met).
- An effective agentic prompt = Role + Task + Reflection/Criteria + Output Format + Constraints — missing any one degrades output quality.

Key terms: CIFC, CICO/SECO, zero-shot/few-shot CoT, self-correction, feedback loop.

---

6. Memory in AI Agents

6.1 Core Concepts

- **State** = an agent's retained memory of past interaction within a session.
- **Stateless agents**: no memory; every request independent (e.g. OTP verification, Google Translate) — simple but repetitive/context-blind.
- **Stateful agents**: retain history → personalization, multi-turn coherence, less repetition.
- **Context** = all stored info for the current session (inputs, prior messages, preferences, decisions).
- **Context window** = the max text (in tokens) a model can process at once; varies by model; when full, oldest messages are evicted **FIFO**. Managing this deliberately is called **context engineering**.

6.2 Short-Term vs Long-Term Memory

	Short-Term (STM)	**Long-Term (LTM)**
Scope	Single session/ conversation	Persists across sessions

	Short-Term (STM)	Long-Term (LTM)
Storage	Inside the context window	External DB (SQL/ NoSQL/vector)
Speed/Cost	Faster, cheaper	Slower, costlier (query/ retrieval overhead)
Use	Active conversation continuity	User preferences, history, embeddings

- **Memory flavors** (apply to LTM): **Episodic** (specific past events, e.g. "user asked for vegetarian options last Tuesday"), **Semantic** (general facts, e.g. drug dosage guidelines), **Procedural** (step-by-step workflows, e.g. booking process).

6.3 STM Management Strategies

Strategy	Mechanism	Trade-off
Buffer memory	Store entire history, pass all of it each turn	Full detail, high token cost
Window memory	Keep only the last *N* messages (`history[-10:]`)	Lightweight, loses early context
Summary memory	Maintain a running LLM-generated summary, clear raw messages after a threshold	Token-efficient, risks losing detail

- Databases for memory: **relational (SQL)** for structured profiles/preferences; **vector DBs** for embeddings/semantic recall; key-value/file stores for simpler needs.
- **Memory flow**: user message → fetch relevant long-term memory → merge with current message as context → generate response → write new interaction back to memory.

Key terms: FIFO eviction, context engineering, buffer/window/summary memory, episodic/semantic/procedural memory.

---

7. Databases for AI Systems

7.1 Why Not CSV/Excel

- Break down at scale (millions of rows); not built for **concurrent** multi-user access — fine only for small, single-user data.

7.2 Database Types

Type	Best For	Notes
Relational (SQL)	Structured, related data (banking, profiles)	Tables + keys; queried with SQL
NoSQL	Unstructured/loosely-related data (social graphs)	Key-value, JSON docs, graphs; no SQL
Vector	Embeddings, semantic search / RAG	High-dimensional similarity search
Time-series	Timestamped data (traffic, sensors)	Anomaly detection, forecasting

7.3 Schema, Keys & Constraints

- **Table** = rows (records) × columns (attributes); **schema** defines column names/types.
- **Primary key**: uniquely identifies a row (unique + non-null). **Foreign key**: references another table's primary key → relational integrity.
- **Constraints**: `NOT NULL`, `UNIQUE`, `PRIMARY KEY`, `DEFAULT`, `CHECK`.
- **SQL data types**: `INTEGER`, `FLOAT`, `VARCHAR` (bounded text), `TEXT` (large text), `BOOLEAN`, `DATE`, `TIMESTAMP`, `UUID` (common PK for large systems).

7.4 SQL Command Reference

Command	Purpose	Example
`CREATE TABLE`	Define a table + constraints	—
`INSERT INTO`	Add row(s)	—
`SELECT`	Retrieve columns	`SELECT * FROM students;`
`WHERE`	Filter rows (`AND/OR/NOT`)	`WHERE age > 25`
`UPDATE ... SET ... WHERE`	Modify matching rows	`UPDATE students SET city='Delhi' WHERE id=5;`
`DELETE FROM ... WHERE`	Remove matching rows	`DELETE FROM students WHERE id BETWEEN 2 AND 5;`
`LIKE`	Pattern match (`%` = 0+ chars, _ = 1 char)	`WHERE name LIKE 'J%'`
`ORDER BY ... ASC/DESC`	Sort results	—
`LIMIT`	Cap row count	`LIMIT 5`

Command	Purpose	Example
`OFFSET`	Skip N rows (pagination, pairs with `LIMIT`)	—
`JOIN ... ON`	Combine tables on a matching key	Inner join = only matches; Cross join = full Cartesian product (m×n, rarely useful)
Table alias	Shorten table names in query	`customers AS c` — temporary, doesn't rename the table

- SQL is **case-insensitive** for keywords; string literals need quotes.
- **Normalization**: splitting one large table into smaller related tables to remove redundancy (e.g. a separate `teachers` table instead of repeating teacher names per row).
- **Relationships**: **1:1** (user ↔ Aadhaar), **1:many** (customer ↔ orders), **many:many** (students ↔ courses, via a junction table).

Key terms: primary/foreign key, normalization, junction table, Cartesian product.

---

8. Vector Databases & Embeddings

8.1 Embeddings

- An **embedding** is a numeric vector encoding the meaning of text (word/sentence/doc). Semantically similar text → nearby vectors (e.g. "dog" close to "Labrador"; "keyboard" far from "mouse").
- Vector arithmetic captures relationships: `king − male + female ≈ queen`.
- Text is **chunked/tokenized** first, then each chunk is passed through an embedding model (OpenAI, Google, BG/Beijing-AI, HuggingFace sentence-transformers, etc.).

8.2 Chunking Strategies

- **Fixed-size**: split by a set size — simplest, least context-aware.
- **Semantic**: group by meaning relatedness.
- **Paragraph-based**: use natural paragraph boundaries.
- **Recursive**: for hierarchical/nested documents (e.g. legal text).
- **Overlap** (e.g. 10–15%) between chunks preserves context across boundaries.
- Chunk size/overlap directly affects retrieval quality — too small fragments information; too large dilutes relevance.

8.3 Vector DB Core Jobs

1. **Storage** — embeddings + metadata (metadata enables filtering, e.g. by category).
2. **Indexing** — organizes vectors for fast lookup (e.g. ANN graphs), like postal codes grouping nearby regions.

3. **Retrieval** — given a query vector, return the top-*k* nearest vectors.

8.4 Similarity Search

- **Cosine similarity** (angle between vectors) is preferred over Euclidean distance for text embeddings — smaller angle = higher similarity.
- **Brute force search**: compares the query to every vector — O(n), accurate but slow at scale.
- **ANN (Approximate Nearest Neighbor)**: clusters/graphs to jump between nearby groups — ~O(log n), trades a little accuracy for large speed gains.

8.5 Popular Vector DBs & Tools

Tool	Notes
ChromaDB	Open-source, widely used in these labs
Pinecone	Paid, managed service
Faiss	Facebook AI, optimized large-scale similarity search
PGVector	PostgreSQL extension, less common here

8.6 ChromaDB Hands-On Reference

Function	Purpose
`get_or_create_collection`	Create/open a collection (≈ SQL table)
`upsert`	Insert or update embeddings + docs + metadata
`count()`	Total records stored
`peek()`	Sample stored records
`get()`	Fetch a record by ID
`query()`	Similarity search, returns top-*k* + distances

- Setup: `pip install chromadb`; persistent client points at a local folder; sentence-transformer models commonly output 384-dim embeddings.

Key terms: cosine similarity, ANN, top-*k*, chunk overlap, persistent client.

---

9. Retrieval-Augmented Generation (RAG)

9.1 Why RAG

- LLMs have static, training-cutoff knowledge and can hallucinate when asked about specifics they were never trained on (company policy, live prices). **RAG grounds** generation in an external, verifiable knowledge base.

9.2 RAG = Retrieval + Augmented + Generation

- **Retrieval**: embed the query, search the vector DB for relevant chunks.
- **Augmented**: combine retrieved chunks with the user query into a prompt.
- **Generation**: the LLM answers using that augmented context.

9.3 RAG Components

Knowledge Base · Embedding model · Vector Database · System Prompt (combines context + query) · **Grounding** (verifying the answer is actually supported by retrieved data — reduces hallucination) · Ingestion Pipeline (scrape → clean → chunk → embed → store) · Scheduler (keeps dynamic data like prices fresh).

9.4 End-to-End Flow (offline + online)

Offline (ingestion): collect data (PDF/URL/DB) → clean/normalize → chunk → embed → store in vector DB (repeat via scheduler for dynamic data). **Online (query)**: user query → embed query → retrieve top-*k* via ANN → construct context+query prompt → LLM generates → **ground** the answer → reply.

9.5 Failure Modes & Diagnosis

- **Wrong chunk retrieved** / **no relevant chunk exists** / **right chunk retrieved but ignored by the LLM**.
- Always diagnose **retrieval failure** (bad chunk) vs **generation failure** (LLM misuses a good chunk) **separately** — different fixes apply.
- Typical RAG accuracy ceiling ≈ 99.5%, not 100%; requires ongoing maintenance (refresh/remove stale embeddings).

9.6 Building a RAG System (worked example: "Shopcart")

- **3 core components**: Knowledge Source (raw policy text) → Retriever (embed + ChromaDB top-*k*) → Generator (LLM answers from retrieved chunks only).
- Libraries: `sentence-transformers` (BG embedding model), `chromadb`, an LLM client (e.g. Groq), `python-dotenv` for `.env` API keys.
- Program flow: load embedding model → init vector store → embed & insert chunks → embed user query → retrieve top-*k* → build system prompt with context → call LLM → return grounded answer (+ debug info: ranks/distances).
- A later pipeline variant used `policy_documents/`, 100-word chunks, default top-k=3, and a system prompt like *"You are a shopping support assistant — answer only from the context block."*

9.7 Improving a Baseline RAG System

- **Strict system prompts**: explicitly forbid invention; require a refusal when data is insufficient.
- **Category/metadata filtering**: tag chunks by category (returns, shipping, warranty…), detect the query's category via keywords, and restrict retrieval to that subset — narrower, more accurate, but can miss multi-category queries (extendable to multi-category retrieval).
- **Fetch more chunks** (e.g. top-5 instead of top-2/3) to raise the odds of covering the answer.
- Category tags must match **case-consistently** (normalize to lowercase).

9.8 Advanced RAG Optimization

Technique	What it does	Trade-off
Query expansion	LLM generates 3–5 phrasings of the query before retrieval	+latency/cost, better recall for vague queries
Compression	A smaller LLM shrinks retrieved context before the final call	Cheaper/faster, risk of losing detail
Re-ranking (cross-encoder)	Retrieve a large pool, re-rank by relevance, keep top few	Preserves all info; industry-preferred over compression
Semantic chunking	Embed sentences, group by similarity rather than fixed size	10–20× slower to build, better retrieval quality
Semantic caching	Cache Q→A pairs with a similarity threshold + TTL	Big latency/cost win for repeat/FAQ-style queries
Intent classification/ routing	Classify query type (factual/exploratory/ comparative/out-of-scope) and route to the matching pipeline	Avoids wasting a full pipeline on simple/out-of-scope queries
Multimodal RAG	Separate embeddings for text, images, tables in a document	More complex, needed for real-world PDFs

- Worked example: a **three-route triage** (Direct Lookup — 5 chunks; Thematic — 40 chunks + re-rank; Strategic — full pipeline + expansion) tuned for a financial-report Q&A bot, with measured cache-hit latency (~43 ms) vs cache-miss (~298 ms).

Key terms: grounding, ingestion pipeline, retrieval vs generation failure, re-ranking, semantic caching.

---

10. APIs, JSON & Tool Calling

10.1 HTTP & JSON Basics

Method	Purpose
`GET`	Retrieve data
`POST`	Create data / trigger an action
`PUT`	Replace a resource fully

Method	Purpose
`PATCH`	Partially update
`DELETE`	Remove data

Status code	Meaning
200 / 201	Success / Created
400	Bad request (client error)
403	Forbidden
404	Not found
429	Too many requests (rate-limited)
500 / 503	Server error

- **JSON**: lightweight key-value data format, mirrors Python dicts; types = string (double-quoted), number, `true`/`false`, array, `null`.
- Python: `json.loads()` (string → object), `json.dumps()` (object → string); `requests` library sends HTTP calls (`pip install requests`); always check status codes and handle timeouts.
- APIs let agents fetch **live data** (weather, order status) vs the **static data** baked into RAG documents.

10.2 Tool Calling Mechanism

1. User query received.
2. LLM decides which tool(s) to call (usually emits a JSON-structured instruction).
3. Program executes the tool function/API.
4. Tool result returned to the LLM.
5. LLM synthesizes the final natural-language answer.

- **Tool schema**: `name`, `description` (routing hint for the model), `parameters` — accurate descriptions are what let the model choose correctly.
- `tool_choice="auto"` lets the LLM decide dynamically whether/which tool to invoke.

10.3 Model Context Protocol (MCP)

- Anthropic standard (2024) that standardizes how agents discover/call tools, replacing bespoke per-API integration code.
- **The N×M problem**: without a standard, *N* clients each needing *M* tools require *N*×*M* custom integrations. **MCP** centralizes this to **N+M** connections via a shared MCP Server.
- **Components**: MCP **Server** (exposes standardized tools, handles auth), **Tools** (discoverable, described capabilities), **Protocol** (standard JSON request/response).
- Minimal server pattern (FastMCP):

```python
from fastmcp import FastMCP
server = FastMCP("MyServer")

@server.tool
def get_menu():
    """Retrieve menu items"""
    return menu_data

server.run()
```

- Real-world MCP servers: Hugging Face (model/space search), Stripe (invoices/refunds/payments), GitHub (code/security scanning), Atlassian (Jira/Confluence).
- A RAG pipeline itself can be wrapped as an MCP tool (`ask_<domain>_question`) so other agents can discover and reuse it.

Key terms: N×M problem, tool schema, `tool_choice`, MCP server.

---

11. Local LLMs with Ollama

- **Ollama**: open-source app to download/run/manage LLMs **locally** — no per-call cost, full data privacy, no internet dependency, but weaker reasoning and no real-time data access; large models need 40 GB+ RAM.

Command	Purpose
`ollama pull <model>`	Download a model
`ollama list`	List downloaded models
`ollama run <model>`	Chat with a model
`ollama rm <model>`	Delete a model
`ollama --version`	Check installed version

- Python: `pip install ollama`; `ollama.chat(model=..., messages=[...])`.
- **Ollama Cloud** offers stronger hosted models (needs an API key + internet) — toggle locally via an env flag like `USE_CLOUD`.
- Practical scope: learning/prototyping, not typical production use.

---

12. LangChain Framework

12.1 Why LangChain

- Manual integration of LLMs with tools/memory/retrieval is verbose and repetitive. LangChain provides ready-made, composable components (like a web framework for AI apps) — reducing hundreds of lines to a handful of function calls.
- Supports swapping LLM providers (Ollama, OpenAI, Groq, Gemini, Claude) with minimal code change.

- Default RAG/chunking/embedding components suit simple use cases; complex/ custom pipelines may still be hand-built.

12.2 Core Components

Prompts (templated instructions) · **Chains** (link components, pass output → input) · **Agents** (decide which tool to call) · **Memory** (recall past turns) · **Retrieval/RAG** · **Tools** (external services) · **Runnables** (composable units).

12.3 Environment & LCEL

- Create isolated environments: `python -m venv env_name`; activate per OS.
- Packages: `langchain-core`, provider packages (`langchain-ollama`, etc.), `python-dotenv`.
- `.env` holds secrets (API keys, model name, host URL); `.env.example` documents required keys without values; `.gitignore` keeps `.env` out of version control; `load_dotenv()` loads it into the OS environment, with code-side defaults if a variable is missing.
- **LCEL (LangChain Expression Language)**: chain components with the pipe operator: `prompt | llm | output_parser`. Swapping the LLM class (e.g. `ChatOllama` → `ChatOpenAI`) is a one-line change.
- **PromptTemplate**: reusable prompt with variable placeholders (topic, audience, tone…) filled at runtime.
- **Temperature**: 0.0–0.3 = focused/predictable; 0.4–0.7 = balanced; 0.8–1.0 = creative/more hallucination risk.

12.4 Tools in LangChain

Function	Purpose
`@tool`	Decorator wrapping a Python function as a callable LangChain tool
`bind_tools()`	Attaches the tool list to an LLM — called as `llm.bind_tools(tools)`
`tool_calls`	List **attribute** on `AIMessage` holding which tool(s) the model wants to call — read directly as `response.tool_calls` (no parentheses)
`ToolMessage(content=..., tool_call_id=...)`	Class constructed directly to wrap a tool's output for the model to read

- Flow: **HumanMessage** → **AIMessage** (names the tool call) → tool executes → **ToolMessage** (result) → LLM produces the final reply.
- Model capability affects reliability: small models (e.g. Gwen) can be inconsistent; larger models (Llama 3.1+, GPT-4, Claude) are more reliable for critical tasks (e.g. financial calculations); lower temperature improves determinism for tool calling.

12.5 Building Agents

- `create_tool_calling_agent(llm, tools, prompt)` — builds an agent that automatically decides and invokes tools (vs the earlier manual-invocation pattern); takes no `memory` argument — passing one raises `TypeError`. Memory is handled separately (e.g. via `AgentExecutor` or `RunnableWithMessageHistory`).
- `agent_executor(agent, tools, max_iterations, verbose, handle_parsing_errors, return_intermediate_steps)` — runs the agent loop safely.
  - `max_iterations`: caps retries (prevents infinite loops); defaults to 3.
  - `return_intermediate_steps`: logs each tool call + result (debugging).
  - `handle_parsing_errors`: catches malformed output gracefully.
  - `verbose`: toggles detailed execution logs.
- If no suitable tool exists, the agent falls back to a direct LLM response.

12.6 Memory in LangChain

- **Manual**: a plain list storing human/AI message turns.
- **Automatic**: `InMemoryChatMessageHistory` (session-keyed dict of message lists), `RunnableWithMessageHistory` (runs the agent while managing that history), `MessagesPlaceholder` (reserves a slot for chat history inside a prompt template), `AgentScratchPad` (stores intermediate tool-execution steps during a single run).
- **Sessions** isolate conversations — switching session IDs resets context, just like separate chats.
- In-memory storage is volatile (lost on restart); production systems persist to PostgreSQL/Supabase/vector DBs.

12.7 RAG + Tools + Memory in LangChain

- Full RAG pipeline: ingest docs (`.md`/PDF) → chunk (e.g. 800 words, ~10–15% overlap) → embed (OpenAI/HuggingFace/BG/Groq-hosted) → store in Chroma/Pinecone → retrieve top-*k* → generate with a source-citing system prompt.
- `RunnablePassthrough` passes the original question through a chain unchanged (e.g. alongside retrieved context).
- Combined agents mix a RAG tool (policy/document Q&A), custom function tools (date conversion, order lookup), and memory — with a system prompt instructing the agent to admit "I don't know" for out-of-domain queries rather than guessing.
- Re-run the ingestion step whenever source documents change (embeddings otherwise go stale).

12.8 Evaluating & Debugging Agents

- Unlike a pure function, an agent must be checked on its **entire trajectory** — correct tools called, correct order, correct final answer — not just the final output.
- **Evaluation cases** (JSON): question, **must-use tools**, **forbidden tools**, expected referenced documents, **should-refuse** flag, expected content substrings.
- **Why not use an LLM as the judge here**: non-deterministic, costly, can hallucinate — deterministic Python checks are preferred for this kind of structural evaluation.

- **Tracing**: `verbose=True` + a global trace object (singleton pattern) records every tool call, retrieval, input/output, and timestamp — described as a "black-box flight recorder" for diagnosing failures.
- Clear chat history **before each** evaluation case to avoid context leaking between tests.
- Debugging chunking: reducing chunk size fragments answers; increasing size/overlap improves retrieval — verify with direct before/after comparisons.

12.9 Real-World Agent Example: HR Assistant

- Answers policy/benefits/IT questions via RAG over markdown documents (leave policy, IT setup, benefits FAQ, org hierarchy).
- **Sensitive-query handling**: detects keywords (salary, CTC, appraisal, promotion) and **refuses**, escalating to a human ("human-in-the-loop") rather than guessing or leaking data.
- **IT ticketing**: auto-creates a support ticket (hashed unique ID) for reported technical issues.
- **Memory**: short-term = session history; long-term = a third-party per-user memory service (e.g. "Mem Zero") persisting preferences across sessions.
- **Security**: session-hijacking risk if session IDs leak from browser storage — mitigated by not storing sensitive data client-side and routing confidential queries to human review.
- **Evaluation metrics**: refusal accuracy, escalation correctness, factual accuracy (vs source docs), hallucination detection.

Key terms: LCEL, `RunnablePassthrough`, agent trajectory, trace/singleton pattern, human-in-the-loop escalation.

---

13. Multi-Agent Systems

13.1 Core Concepts

- **Single agent**: one entity plans + executes everything — simpler, but a bottleneck for complex work.
- **Multi-agent**: an **orchestrator** decomposes work across **specialized sub-agents** (each with a narrow role, possibly a different LLM), running sequentially or in parallel.
- **Sequential workflow**: strict order (e.g. Research → Write → Edit).
- **Collaborative/parallel workflow**: independent sub-tasks run simultaneously, faster overall (e.g. research + budgeting + itinerary for trip planning).
- **Agent-to-Agent (A2A) communication**: user only talks to the orchestrator; orchestrator delegates, sub-agents report back, orchestrator assembles the final output.

13.2 APIs vs Webhooks

- **API (request-response)**: client actively pulls data from a server.

- **Webhook (event-driven)**: server automatically pushes data to the client when an event fires (e.g. a price-drop alert). **Triggers** can be time-based (every hour) or condition-based (price < threshold).

13.3 No-Code: N8N

- Drag-and-drop workflow automation; supports 1500+ integrations, Python/JS scripting nodes, and multiple LLM providers (Groq, Gemini, OpenAI, Ollama, Claude).
- **Deployment**: Cloud (no install, 14-day free trial) vs Local/Docker (heavier setup, better performance under load).
- **Node types**: Trigger (Manual, Schedule, Chat, Webhook, Form, third-party, or another workflow), Action (Gmail, Sheets, Slack…), LLM/AI (the "brain"), Logical (if-else branching), Expression (custom JS/Python, `${}` variable syntax).
- **Basic LLM Chain** (single model + chat, one tool at most) vs **AI Agent Node** (multi-tool, optional memory, more complex decisions).
- **Memory in N8N agents**: without it, no context across turns; with it, the agent retains a specified number of prior messages.
- **System vs user prompts** (same priority rule as §5.1): system prompts are given priority by design, but a user can still bypass them via prompt injection, so guardrails are still needed — not an absolute security guarantee.
- **Credentials**: stored securely in a dedicated section, referenced via `$` variable notation, reusable across workflows; delete test API keys after use.
- **Practical build pattern**: write a `problem_statement.md` → generate `architecture.md` → generate a phase-wise `implementation_plan.md` → implement phase by phase with an AI coding tool (Cursor/Claude/Antigravity) → test/debug each phase → re-upload the workflow JSON. This "context engineering" approach (detailed docs → better AI-generated code) recurs across several labs.
- Realistic expectation: even a modest end-to-end workflow (e.g. a weather agent) commonly takes several hours of iteration; AI tools cannot anticipate every edge case.

13.4 CrewAI

- Higher-level multi-agent framework (vs N8N's single-agent drag-and-drop model); agents can interact and share context.
- **Agent** = Role + Goal + Backstory. **Task** = Description + Expected Output + (optional) Output File + Context (dependency on prior task output) + Agent Assignment. **Crew** = agents + tasks + a **Process** type, run via `kickoff()`.
- **Process types**: **Sequential** (linear handoff) vs **Hierarchical** (a manager/orchestrator agent delegates, allows parallel work).
- `allow_delegation=True/False` controls whether an agent may hand off work it can't complete.
- Supports per-agent LLM choice and `verbose` logging.

- **Scale guidance**: CrewAI suits 6–10 agents; 500+ agents → AutoGen; deep hierarchical control → LangGraph.
- **Limitations**: limited access to raw system prompts, harder to debug (high abstraction), not considered enterprise-production-grade.

13.5 AutoGen (Microsoft)

- Sits between LangChain (low-level, full control) and CrewAI (very high-level) in abstraction.
- **Core agents**: **User Proxy** (represents the human, can execute functions, sets iteration caps), **Assistant** (the reasoning "brain," suggests tool use), **Conversable Agent** (customizable specialized role, e.g. diagnostic/compliance/risk).
- **Patterns**: Two-agent (Diagnostic → Resolution), Three-agent (+ Escalation for critical issues), Multi-specialist **Group Chat** coordinated by a **Group Chat Manager** (with `max_rounds` cap).
- **Triage/routing**: cheap keyword/semantic/fuzzy matching filters simple queries before invoking an LLM, saving cost.
- Use cases: IT helpdesk, contract compliance/risk review, customer support tiering.

13.6 LangGraph

- Lower-level, most controllable framework — for production multi-agent systems needing fine-grained flow control.
- **Three core components**: **Nodes** (Python functions doing the work), **Edges** (sequential / parallel / conditional connections), **State** (shared working memory that accumulates and flows through nodes — like a patient folder passed department to department).
- **Checkpointing**: saves state snapshots so a workflow can resume after failure/interruption instead of restarting; **Thread ID** links checkpoints to one run. Backends: Memory Saver (in-RAM) vs SQLite Saver (persisted to disk).
- **Human-in-the-loop**: `interrupt()` pauses execution at an approval gate; a **resume** call (same thread ID) continues afterward.
- **Timeouts**: per-node (prevents one slow step blocking everything) and global (caps total run time).
- **Retries**: exponential backoff (e.g. retry after 1s, then 5s, then 20s) for transient failures like rate limits.
- **Design principle**: one node = one clear responsibility; don't over-fragment or over-consolidate; use cheaper models for routing/classification nodes, stronger models for actual reasoning nodes.
- **Worked example — Healthcare Assistant (7 nodes)**: Generate SQL → Classify query (READ/WRITE/UNSAFE) → HITL approval gate for WRITE → Execute → Format results → Handle rejection → Audit log. READ auto-executes; WRITE needs human sign-off; UNSAFE (e.g. `DROP`) is blocked immediately.

13.7 ReAct Framework & Multi-Tool Agents

- **ReAct** = **Rea**son + **Act**, the iterative loop underlying most agent frameworks: **Thought** (what's needed) → **Action** (call a tool) → **Observation** (read the result) → repeat until done. Each cycle consumes tokens (cost/latency trade-off).
- **Worked example — e-commerce support agent** with 4+ tools: SQL tool (customers/orders/refunds via NL→SQL), RAG tool (policy documents), Feedback tool (logs ratings), Defer-to-Human tool (escalates, logs to a deferred-cases table).
- **Agentic RAG**: wrapping RAG itself as one tool among several, with hard rules — refuse out-of-scope questions, always cite sources, explicitly say "I don't know" rather than guess.
- **No-code caution**: granting broad edit-level credentials to third-party no-code platforms (e.g. Make.com) is a real security risk; prefer trusted/established platforms and least-privilege access.

Key terms: orchestrator/sub-agent, A2A, checkpointing, thread ID, exponential backoff, ReAct loop, agentic RAG.

---

14. Production, LLM Ops & Deployment

14.1 FastAPI for Production Backends

- Gradio/Streamlit suit demos; **FastAPI** (or Flask) is the production standard for exposing a backend.
- `FastAPI()` bootstraps an app; `@app.get()`/`@app.post()` turn functions into live endpoints; **lifespan** manages startup/shutdown (setup before `yield`, cleanup after); requests auto-validate against defined models; Swagger docs auto-generate at `/docs`.
- **GET** = retrieve; **POST** = send/create data (e.g. submit a RAG question).
- Frontend (typically React/Next.js in production, not Python) and backend communicate purely via JSON over HTTP — fully decoupled, framework-agnostic.

14.2 Deployment Architecture

- **Frontend hosting**: Streamlit (quick/basic) or Vercel (React/Next.js).
- **Backend hosting**: Railway/Render, containerized with **Docker** for consistent dependencies.
- **Ingestion pipeline**: a separate scheduled job (e.g. GitHub Actions) re-embeds documents and updates the vector DB (e.g. Pinecone) whenever source docs change — decoupled from the live query path.
- **Flow**: frontend → backend API → retrieval + LLM inference → response back to frontend; ingestion runs independently on its own schedule.

14.3 LLM Ops

- Bridges notebook-stage development (building RAGs/agents) to reliable production operation: **versioning, deployment, evaluation, observability, security, release control**.
- **Versioning**: track system prompts/configs centrally (not as throwaway notebook variables) with a full iteration audit trail.
- **Observability/Traces**: a trace records the full path of one interaction — input, retrieved context, prompts, model/tool calls, output, latency, token usage — essential for audits and compliance (e.g. proving what a finance chatbot told a user).
- **Evaluation via LLM judges**: one LLM scores another's output across **Groundedness** (supported by retrieved context?), **Relevance** (answers the actual question?), **Context Relevance** (was the retrieved context appropriate?). Judges must first be validated against 100–300 human-labeled gold examples, watching especially for false positives; in production, judges typically sample only a small fraction of live traffic (cost).
- **MLflow**: open-source LLM Ops platform — Experiment Tracking, Prompt Registry (versioned prompts with lineage, like git for prompts), Model Registry, Traces Dashboard (latency/error/token trends), AI Gateway (routes requests across providers by cost/complexity). Alternative: **LangSmith**.

14.4 Guardrails

- **Input-level (pre-processing)**: block toxic/abusive language, prompt-injection attempts, and clearly harmful requests before they reach the LLM.
- **Output-level (post-processing)**: check for toxicity, bias, PII leakage (names/addresses/phone numbers), internal-policy leakage, inconsistency across similar queries, and factual accuracy before showing the response to the user.
- **Implementation approaches**: strengthen the system prompt with explicit safety instructions (e.g. redact PII with XXX); dedicated classifier prompts (e.g. "Is this a prompt injection? yes/no"); libraries such as **Detoxify** (toxicity scoring) and **LLM Guard** (injection/toxicity/bias/output validation bundle).
- **Worked 4-phase rollout** (e-commerce chatbot): Phase 1 baseline (leaks PII/policy, inconsistent) → Phase 2 stronger system prompt (confidentiality, consistency) → Phase 3 input sanitization (blocks injection/toxicity) → Phase 4 output sanitization (masks sensitive data, checks consistency).

Key terms: trace, LLM judge, gold examples, prompt injection, PII, AI Gateway.

---

15. Cost & Model Selection Framework

15.1 Ten Business Dimensions (drive model choice — not price first)

1. **Failure cost** — financial/real-world impact of a wrong answer.

2. **Reversibility** — can an error be caught/fixed before harm (e.g. edit a draft) or is it permanent (e.g. a wire transfer)?
3. **Volume** — call frequency at scale (favors cheaper models for very high volume).
4. **Criticality** — is this function on the product's critical path?
5. **Latency sensitivity** — is a human waiting live, or can it run as a batch job?
6. **Output consumer** — machine, general public, or a vulnerable population (minors/elderly) needing stronger safety guardrails?
7. **Input quality** — clean/structured input allows a simpler model; noisy/multimodal input needs a stronger one.
8. **Change frequency** — how often do workflows/expectations shift?
9. **Auditability** — must you show a decision trail (finance/government)?
10. **Error ownership** — does the company or the end-user bear the cost of a mistake?

- **Layering pattern**: use a cheap/fast model (e.g. Haiku) for high-volume initial filtering, escalate only shortlisted cases to a stronger model (Sonnet), reserve the most powerful tier + human review (Opus) for the final, highest-stakes decision (e.g. resume screening funneled thousands → hundreds → 10 → 1 hire).

15.2 Cost Optimization Levers

- **Batch API**: ~50% discount for non-real-time workloads (24h turnaround acceptable) — good for reports, bulk scoring.
- **Prompt caching**: ~90% discount on repeated/stable prompt prefixes (system instructions, schemas, policies); short TTL (minutes–hour); not portable across providers/models.
- **Extended thinking**: adds reasoning tokens (higher cost) but raises accuracy for high-stakes decisions — budget for it explicitly.

15.3 Cost Formula & Buffers

- **Cost per request** = (input tokens × input price) + (output tokens × output price).
- Always budget for: a **failure buffer** (10–15%, for retries/validation failures), token-growth headroom, and the fact that **output tokens typically cost ~5×** input tokens — so trimming completion length matters more than trimming input.
- Common mistake: costing only the "happy path" and ignoring retries, growth, and output-heavy pricing.

15.4 Framework Summary

Wrong order: Cost → Model → Benchmarks. **Right order**: define the business outcome → quantify failure cost → assess the ten dimensions → pick a model tier → apply Batch API/caching → compute unit economics → validate against benchmarks.

Key terms: unit economics, Batch API, prompt caching, extended thinking, failure buffer.